

Die schwerwiegendsten Funde betreffen Codeausführung in der Bedienoberfläche, offengelegte Konfigurationen und Datenverlust beim Deploy beziehungsweise Zurücksetzen. Die Behebungen können innerhalb der bestehenden Architektur erfolgen.

ENTWICKLER-CHECK.md habe ich zuerst gelesen. Die zentralen Befunde habe ich mit isolierten Originalfunktionen, temporären Git-Repositories und lokalen rsync-Aufrufen überprüft. KI-Antworten wurden simuliert; ein vollständiger Browser-/Express-Integrationstest und echte Hosting-/API-Aufrufe erfolgten nicht. Alle Zeilenangaben beziehen sich auf den unveränderten ZIP-Stand.

1. Kritisch – Präparierte Projektdaten ermöglichen XSS in der Bedienoberfläche

Dateien: `<> ui/app.js , Z. 1511; <> lib/einbacken.js , Z. 91; <> lib/build.js , Z. 230.`

Ein Schlüssel in `.media-edits.state.json` kann HTML enthalten. Dieses gelangt über eine Fehlermeldung unverändert in `b.hinweise` und anschliessend in `innerHTML`. Ein harmloser Test mit `<img src=x onerror=...>` erreichte diese Ausgabestelle. Damit läuft eingeschleuster Code auf Port 4400 und kann die Verwaltungs-API bedienen.

Behebung: Hinweise ausschliesslich mit `textContent` beziehungsweise Textknoten darstellen. Überschrift und Zeilenumbrüche separat als DOM-Elemente erzeugen.

2. Kritisch – Die Vorschau legt Konfigurationsdateien offen

Datei: `<> server.js , Z. 1512, 1547, 1560, 1579, 1613.`

`/projekt/kalender/config.php/` umgeht die PHP-Prüfung: Wegen des abschliessenden Slash passt `\.php$` nicht. Danach entfernt die Pfadaufbereitung den Slash und reicht `/kalender/config.php` an den Dateiserver weiter. Das wurde für Quelle und Build bestätigt. Zusätzlich erlaubt `dotfiles: 'allow'` beispielsweise das direkte Lesen vorhandener `.env`-Dateien.

Behebung: Den vollständig normalisierten Zielpfad vor jeder Auslieferung prüfen. Versteckte Dateien standardmässig sperren und benötigte Sidecars ausdrücklich erlauben; bekannte Geheimnisdateien ebenfalls sperren.

3. Kritisch – `rsync --delete` löscht ausschliesslich auf dem Server vorhandene Dateien

Datei: `<> lib/deploy.js , Z. 26, 40, 42.`

Ausgenommen ist lediglich `.build-bericht.json`. Eine separat auf dem Hosting abgelegte `kalender/config.php`, Uploads oder andere Laufzeitdaten verschwinden, wenn sie im synchronisierten Ziel liegen und im Build fehlen. Die Löschung einer solchen Konfiguration wurde lokal reproduziert. Die Zielprüfung akzeptiert ausserdem gefährlich breite Ziele wie `.` und `/`.

Behebung: Serverdateien und Laufzeitverzeichnisse ausdrücklich durch rsync-Schutzfilter erhalten. Nur dedizierte Deploy-Verzeichnisse zulassen und vor destruktiver Synchronisierung eine Zielmarkierung prüfen. [rsync-Dokumentation](#)

4. Kritisch – Zurücksetzen kann ignorierte Dateien unwiederbringlich löschen

Datei: `</> lib/git.js`, Z. 234, 240, 241, 246.

Zuerst wird die alte `.gitignore` wiederhergestellt, danach läuft `git clean -fdq`. Eine bisher ignorierte Datei kann dadurch ihren Schutz verlieren. Im Test verschwand eine nie eingecheckte Datei aus `private/` endgültig, weil der Zielstand diese Ignore-Regel noch nicht enthielt. Die vorherige Sicherung hatte sie ebenfalls ignoriert.

Behebung: Vorher die aktuell versionierten Pfade erfassen und Löschungen auf diese Menge beschränken. Ignorierte Dateien und Kollisionen mit ihnen separat schützen; die aktuellen Geheimnis-Ausschlussregeln erhalten. [Git-Dokumentation](#)

5. Kritisch – Erkannte Geheimnisse werden trotzdem an die KI übergeben

Dateien: `server.js`, Z. 230, 238, 355, 359; `</> lib/endpruefung.js`, Z. 155, 177.

Hauptkontext und Nachreichrunde prüfen nur den Dateinamen. Ein Testschlüssel in `settings.js` wurde von der Analyse erkannt, aber dennoch vollständig in den KI-Kontext aufgenommen. Auch die Endprüfung filtert ihre ausgewählten Code-Dateien nicht anhand des Geheimnisinhalts.

Behebung: Eine gemeinsame Inhaltsprüfung unmittelbar vor jeder Übergabe an die KI verwenden – für initialen Kontext, Nachreichen, Endprüfung und SEO-Hilfe. Betroffene Dateien vollständig auslassen und dies ohne Schlüsselwert melden.

6. Kritisch – Neue Geheimnisse können automatisch im Git-Fernlager landen

Dateien: `lib/git.js`, Z. 166, 178; `server.js`, Z. 428, 431, 1233.

`sichern()` führt ungeprüft `git add -A` aus und startet anschliessend den automatischen Push. Neu hinzugekommene Geheimnisse werden nicht vorab erkannt. Bei übernommenen Änderungen läuft die Analyse erst nach dem Commit. Im Test wurde eine nachträglich hinzugefügte `later.js` mit Testschlüssel eingecheckt.

Behebung: Geänderte und neue Dateien vor dem Staging frisch auf Geheimnisse prüfen. Ausschlussregeln vor dem Commit aktualisieren und bei einem Fund den betroffenen Commit beziehungsweise Push verhindern.

7. Kritisch – Asset-Downloads ermöglichen Zugriffe auf lokale und interne Dienste

Datei: `lib/build.js`, Z. 97, 99, 104, 281.

URLs aus einem fremden Export werden serverseitig ohne Netzwerkzielprüfung geladen, einschliesslich Weiterleitungen. Im Test wurde die JSON-Antwort eines Loopback-Dienstes als öffentliches Build-Asset gespeichert. Damit können auch lokale Verwaltungs-APIs oder interne Dienste angesprochen und deren Antworten später veröffentlicht werden.

Behebung: Nur öffentliche HTTP(S)-Ziele zulassen, aufgelöste IPv4-/IPv6-Adressen und jedes Weiterleitungsziel prüfen. Antwortgrössen begrenzen und unerwartete Inhaltstypen ablehnen. OWASP-Empfehlungen zu SSRF

8. Kritisch – Vorschau-Schreibrechte sind weder an Projekt noch Bearbeitungsmodus gebunden

Datei: `server.js`, Z. 1439, 1450, 1462, 1494.

Die Endpunkte vertrauen dem übergebenen Projektnamen. Skripte aller Projekte teilen dieselbe Origin auf 4401. Auch eine Build-Seite kann die Schreibendpunkte direkt aufrufen; das Weglassen der Brücke erzwingt keinen Schreibschutz. Eine HTML-Änderung mit eingefügtem `<script>` in einem anderen Projekt wurde vom Originalhandler akzeptiert.

Behebung: Schreibaktionen über die vertrauenswürdige UI vermitteln. Dort Absenderfenster, ausgewähltes Projekt und Quellmodus prüfen; die Serverfreigabe mit einem nur der UI zugänglichen Token absichern. Beliebige HTML-Änderungen dürfen nicht als reine Textbearbeitung durchgehen.

9. Kritisch – Die Port-Trennung verhindert keine unautorisierten Deploy-POSTs

Datei: `server.js`, Z. 60, 811, 1076.

Schreibende Verwaltungsrouten prüfen weder `Origin` noch ein Sitzungstoken. Ein Vorschau-Skript kann einen einfachen POST ohne JSON-Body an Build oder Staging senden. Bei bekanntem Projekt und eingerichtetem Ziel lässt sich dadurch eine Veröffentlichung auslösen. CORS verhindert dabei das Lesen der Antwort, aber nicht grundsätzlich das Absenden solcher Anfragen. MDN zur Same-Origin-Policy

Behebung: Für sämtliche schreibenden Verwaltungsrouten die exakte UI-Origin und ein Sitzungstoken verlangen. Zusätzlich den erwarteten `Host` prüfen. Dafür ist kein Benutzerkonto-System erforderlich.

10. Mittel – Die Warteschlangen serialisieren zusammengehörige Vorgänge nicht

Dateien: `server.js`, Z. 811, 854, 945, 1076; `lib/projects.js`, Z. 70.

Build verwendet `id`, Deploy `deploy:id` und Endprüfung `pruefung:id`. Sie laufen deshalb gleichzeitig auf denselben Dateien. Ein Build kann während rsync das Build-Verzeichnis löschen. Mehrere Metadatenänderungen umgehen die Schlange vollständig und können sich überschreiben. Auch zwei parallele Import-ID-Abfragen lieferten im Test dieselbe Kennung.

Behebung: Zusammengehörige Schreibvorgänge einschliesslich Lesen und Speichern von `projekt.json` unter denselben validierten Projektschlüssel stellen. Importverzeichnisse atomar reservieren. Lange Prüfungen an einen unveränderlichen Snapshot binden.

11. Mittel – Präparierte ZIPs können Speicher und Datenträger erschöpfen

Dateien: `lib/importer.js`, Z. 56, 59, 93; `lib/vergleich.js`, Z. 43, 74.

Die 500-MB-Uploadgrenze begrenzt nur die komprimierten Daten. Grenzen für Eintragsanzahl, einzelne entpackte Dateien und gesamte Entpackgrösse fehlen. `getData()` materialisiert den vollständigen Eintrag im Speicher. Ein ZIP-Bomben-Stresstest wurde nicht durchgeführt; die fehlenden Grenzen sind im Code erkennbar.

Behebung: Konfigurierbare Entpackgrenzen vorab und während der Verarbeitung durchsetzen. In temporäre Verzeichnisse importieren und fehlgeschlagene Importe vollständig aufräumen.

12. Mittel – Pfadprüfungen verhindern keine Symlink-Ausbrüche

Dateien: `lib/aenderungen.js`, Z. 132, 140; `lib/projects.js`, Z. 28.

`path.resolve()` prüft den geschriebenen Pfad, folgt aber keinen Symlinks. Ein vorhandener Link aus manueller Arbeit oder Git-Synchronisierung kann Lese- und Schreibzugriffe aus dem Projekt herausführen. Das wurde mit `alias/page.html` reproduziert. Zusätzlich akzeptiert `basename()` die Projektkennung `..`.

Behebung: Projektkennungen strikt validieren. Symlinks in Projektwurzeln und Zielpfaden ablehnen oder reale Zielpfade samt Elternverzeichnissen auf

Zugehörigkeit prüfen. Der ZIP-Importer selbst erzeugt durch sein `writeFile()` keine ZIP-Symlinks.

13. Mittel – Dateiumbenennungen überschreiben bestehende Build-Dateien

Datei: `lib/build.js`, Z. 313, 358, 362.

Die Kollisionsprüfung berücksichtigt nur Dateien, die umbenannt werden.

Unveränderte Zieldateien fehlen. Im Test überschrieb `a!.html` beim Umbenennen nach `a.html` die bereits vorhandene andere Seite – ohne Warnung.

Behebung: Die endgültigen Zielnamen aller Dateien vor dem ersten Umbenennen berechnen. Bei Kollisionen abrechnen oder eine ausdrücklich aufgelöste Zuordnung verlangen.

14. Mittel – Der Link-Nachzug verändert externe URLs und verliert Ordnerbezüge

Datei: `lib/build.js`, Z. 307, 339, 345, 382.

Die Ersetzungen arbeiten mit blossen Dateinamen. Im Test wurde

`https://external.example/a.html` zu `https://external.example/a`. Gleichzeitig blieb `Assets/logo.svg` im HTML stehen, obwohl die Datei auf dem case-sensitiven Testsystem nach `assets/logo.svg` verschoben wurde.

Behebung: Lokale Verweise relativ zu ihrer jeweiligen Quelldatei auflösen und anhand vollständiger Pfadzuordnungen ersetzen. Externe URLs unverändert lassen; Query und Fragment erhalten.

15. Mittel – Die HTML-Ersetzungen behandeln Anführungszeichen und Attribute falsch

Datei: `lib/einbacken.js`, Z. 95, 115, 128, 188, 193.

Bei `` erzeugte die Ersetzung im Test

`src="replacement.jpg'art.jpg"`. Das Muster beendet den Wert beim Apostroph, obwohl das Attribut doppelt zitiert ist. `[^>]*` scheitert zudem an `>` innerhalb eines Attributwerts; `\bid` kann auch `data-id` treffen. Eingesetzte URLs werden nicht attributgerecht maskiert.

Behebung: Die betroffenen Tags mit einem zuverlässigen Attributparser bearbeiten, Werte maskieren und eindeutige Attribute erkennen. Die für Editor-Indizes benötigte Zuordnung ausdrücklich erhalten.

16. Mittel – Seitenspezifische Sidecars überschreiben Bilder anderer Seiten

Datei: `lib/einbacken.js`, Z. 185, 189, 210.

`slotSrcSetzen()` durchsucht für jede Sidecar alle HTML-Seiten. Haben `index.html` und `about.html` jeweils `id="hero"`, werden beide durch beide seitenspezifischen Sidecars verändert. Im Test zeigten anschliessend beide Seiten das Index-Bild.

Behebung: Seitenspezifische Sidecar-Dateien ihrer Quellseite zuordnen und Änderungen auf diese Seite beschränken. Globale Sidecars gesondert behandeln.

17. Mittel – Der Staging-Schutz kann trotz Fehler Erfolg melden

Datei: `lib/deploy.js`, Z. 49, 51, 53.

Durch die Verkettung `cd ... && ... && grep ... || printf ...` wird auch nach fehlgeschlagenem `cd` der letzte `printf` ausgeführt. Im Test entstand `.htaccess` im falschen Verzeichnis, während der Prozess mit Exitcode 0 endete. Zudem besteht zwischen rsync und Schutzsetzung ein ungeschütztes Zeitfenster.

Behebung: Den Staging-Schutz vor der Übertragung im Staging-Ausgabestand erzeugen. Bleibt der SSH-Schritt bestehen, nach fehlgeschlagenem `cd` sofort abbrechen und die Headerprüfung ausdrücklich als Bedingung formulieren.

18. Mittel – Die Endprüfung wird dem falschen Stand zugeordnet

Dateien: `lib/endpruefung.js`, Z. 237; `lib/fortschritt.js`, Z. 127.

Gespeichert wird der aktuelle Git-Hash am Prüfungsende, unabhängig davon, welcher Build geprüft wurde. Reproduziert: Build A erstellen, Quelle B speichern, alten Build A prüfen, danach B bauen. B wurde weiterhin als bestanden angezeigt. SEO-Änderungen ausserhalb des Quell-Repositories verschärfen das Problem.

Behebung: Einen Fingerabdruck des tatsächlich geprüften Builds speichern und bei jedem neuen Build vergleichen. Während der Prüfung muss dieser Build unverändert bleiben.

19. Mittel – Unbrauchbare KI-Prüfantworten gelten als erfolgreiche Prüfung ohne Funde

Datei: `lib/endpruefung.js`, Z. 184, 195.

Fehlendes oder ungültiges JSON wird in `[]` umgewandelt. Zwei simulierte Antworten ohne verwertbares JSON führten zu null KI-Funden und keinem Fehlerstatus.

Behebung: JSON und Antwortschema prüfen. Bei ungültiger Antwort die betreffende Prüfphase als fehlgeschlagen oder unvollständig markieren und keinen bestandenen Gesamtstatus erzeugen.

20. Mittel – Verbindungsabbruch stoppt die Endprüfung nicht

Dateien: `server.js`, Z. 841, 855, 1033; `lib/endpruefung.js`, Z. 177.

Die Endprüfung besitzt keinen Abbruchcontroller und reicht kein Signal an `chatStreamen()` weiter. Nach dem Schliessen der Verbindung liefen im Test Prüfung und Speicherung weiter. Bei der SEO-Hilfe verhindert das Abbruchflag lediglich weitere Schleifendurchläufe; der laufende KI-Aufruf bleibt aktiv.

Behebung: Wie beim Chat `res.close` mit einem `AbortController` verbinden, das Signal durchreichen und auch vor dem Start wartender Aufgaben prüfen. SSE-Schreibversuche nach Verbindungsende unterlassen.

21. Mittel – Git-Fehler geben das GitHub-Token an das Frontend weiter

Dateien: `lib/git.js`, Z. 297, 360; `server.js`, Z. 577.

Das Token steckt Base64-kodiert in einem Kommandozeilenargument. Bei einem fehlgeschlagenen `execFile()` enthält `e.message` das vollständige Kommando; `/fernlager/abgleichen` reicht diese Meldung unverändert weiter. Das wurde mit einem künstlichen Token bestätigt. Base64 ist reversibel.

Behebung: Authentifizierungswerte aus der Kommandozeile entfernen, beispielsweise über temporäre Prozesskonfiguration übergeben. Git-Fehler zentral bereinigen und niemals vollständige Kommandos mit Authentifizierungsdaten zurückgeben.

22. Niedrig – Leere Git-Zielstände hinterlassen einen teilweise veränderten Index

Datei: `lib/git.js`, Z. 240, 241.

Bei einem Zielcommit ohne Dateien scheitert `checkout`, nachdem der Index bereits geleert wurde. Im Test blieben staged Löschungen und unversionierte Arbeitskopien zurück.

Behebung: Leere Zielbäume vorab erkennen und kontrolliert behandeln; bei Fehlern den gesicherten Ausgangszustand des Index wiederherstellen.

23. Niedrig – Direkte Textänderungen interpretieren `$`-Ersetzungsmuster

Datei: `server.js`, Z. 1494.

`inhalt.replace(alt, neu)` interpretiert beispielsweise `$&` als alten Treffer. Der Nutzer erhält dadurch anderen Inhalt als eingegeben; weitere Ersetzungsmuster können Dokumentteile duplizieren.

Behebung: `inhalt.replace(alt, () => neu)` verwenden.

24. Niedrig – Unterstützte Sidecar-Namen werden beim Schreiben abgewiesen

Dateien: `server.js`, Z. 1445; `lib/einbacken.js`, Z. 174.

Der Leser unterstützt Unterstriche im Seitensuffix, der Schreibendpunkt nicht.

Beispielsweise wird `.image-slots-team_de.state.json` mit 403 abgewiesen.

Behebung: Eine gemeinsame Dateinamenprüfung für Brücke, Schreibendpunkt und Einbacken verwenden; die Beschränkung auf den Projektstamm beibehalten.

Gut gelöst sind der lexikalische Zip-Slip-Schutz, der `.git`-Importfilter und die Schlüsselablage mit erzwungenem Modus `0600`. Der Chat verwendet korrekt `res.close`, reicht das Abbruchsignal weiter und begrenzt die Nachreichrunde. Auch die normale Git-Rücksetzung mit einer zuvor unversionierten Datei und einer 1-MB-Binärdatei funktionierte im Test.

</> `app.js`
Code

</> `einbacken.js`
Code

</> `build.js`
Code

</> `server.js`
Code

</> `deploy.js`
Code

</> `git.js`
Code

</> `endpruefung.js`
Code

 projects.js
Code

 importer.js
Code

 vergleich.js
Code

 aenderungen.js
Code

 fortschritt.js
Code

Weniger anzeigen

Ist dieses Gespräch bisher hilfreich?